



PowerShell PCI Tool (No Driver)

PCI EXPRESS INVENTORY & LINK DIAGNOSTICS

winlspci User's Manual

`lspci` for Windows — without a kernel driver

A PowerShell tool that enumerates every PCI and PCI Express device in a Windows machine, reports negotiated *and* maximum link speed and width, draws the real bus topology, and answers questions at the attribute level — with no driver to sign, install, or trust.

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI
Express Root Port #6 [8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold
P31/BC711/PC711 NVMe SSD [1c5c:174a] (rev 00)
```

Document revision 1.0 · August 2026 · Requires Windows PowerShell 5.1 · MIT Licence

Contents

1	About winlspci	2
2	What winlspci can and cannot report	3
3	Requirements and installation	4
4	Quick start	5
5	Command reference — standard lspci flags	7
6	Command reference — flags beyond lspci	8
7	Attribute-level queries	9
8	Delimited output and coming from Linux	10
9	Topology	11
10	The PCI ID database	12
11	Design principles	13
12	Performance	13
13	Exit codes and scripting	14
14	Troubleshooting	15
15	Tests	16
A	Appendix A — Flag quick reference	16
B	Appendix B — Glossary	17
C	Appendix C — Licence and credits	18

A NOTE ON NAMES

winlspci is the name of the tool. The command you type is `lspci`, deliberately, so that muscle memory and existing notes carry over from Linux. Where this manual writes `lspci` in an example, that is the literal command; where it writes `winlspci`, it means the tool as a whole — the module, its cmdlets and its data files.

NO KERNEL DRIVER

WINDOWS POWERSHELL 5.1

OFFLINE PCI.IDS

REAL TOPOLOGY

TYPED OBJECTS

JSON / CSV

MIT

1 About winlspci

winlspci answers the question every bench engineer asks first: *what is actually in this machine, and is it running the way it should be?* It does that on Windows, from a normal PowerShell prompt, with nothing installed into the kernel.

1.1 What it is

winlspci is a Windows PowerShell implementation of the familiar Linux `lspci` utility. It lists PCI and PCI Express devices with their vendor, device, subsystem and class identifiers, resolves those identifiers to human-readable names from a bundled copy of the PCI ID database, reports link speed and width — both what the link negotiated and what it is capable of — and can draw the bus as a real parent/child tree.

Beyond that, it does something `lspci` does not: it exposes every field it knows as a typed, queryable attribute, so you can ask precise questions without piping text through a search tool that Windows does not have.

1.2 Why there is no driver

This is the most important thing to understand about the tool, and the reason its capabilities are shaped the way they are.

THE CONSTRAINT

Windows exposes no userland path to PCI configuration space. A faithful port of `lspci` is therefore impossible without a signed kernel-mode driver — and shipping one of those means driver signing, Secure Boot friction, and a considerably larger attack surface on a lab machine.

Rather than accept that cost, winlspci reads Windows' own PnP/PCI enumeration data instead. That data is populated by the Windows PCI bus driver, which reads configuration space itself at enumeration time. The consequence is a clean trade: everything the bus driver chose to surface is available instantly and safely; everything it did not surface is genuinely out of reach, and winlspci says so rather than guessing.

1.3 Design philosophy

One idea runs through the whole tool, and it is worth stating before the reference chapters:

THE GOVERNING RULE

A wrong answer that looks right is worse than an obvious failure.

That principle is why a value the hardware did not report prints as *not reported* rather than as zero; why a filter that matches nothing returns a non-zero exit code instead of an empty-but-successful result; and why asking for a configuration-space dump produces a clear refusal instead of a partial answer dressed up as a complete one. Chapter 11 covers each of these in detail.

1.4 Who it is for

Hardware validation

Confirm a card enumerated at all, at the right slot, with the right IDs — then confirm the link trained to the speed and width the design calls for.

Field and support

Capture a complete, structured inventory of an unfamiliar machine to CSV or JSON with one command, with no software to install on a customer's system.

Linux engineers on Windows

Keep the flags, the output shape and the reflexes you already have, without pretending Windows has `grep`, `awk` or `cut`.

2 What winlspci can and cannot report

Read this chapter before the reference chapters. The boundary below is not a list of features that have not been written yet — it is the boundary of what any driverless tool on Windows can see.

REPORTED	CANNOT BE REPORTED
Device presence; vendor, device and subsystem IDs; revision	Configuration-space hex dumps (<code>-x</code>)
Class, subclass and prog-if, with resolved names	Capability structure walks
<code>bus:device.function</code> address	ASPM state, LTR, DPC
Negotiated and maximum link speed and link width	AER register detail (presence only is reported)
Max Payload Size (MPS) and Max Read Request size (MRRS)	Anything requiring a live register read
Driver binding and driver version; device status and problem code	
NUMA node; AER capability presence	
Bus topology (parent/child, via <code>DEVPKEY_Device_Parent</code>)	
Every field above, queryable per attribute	

2.1 What happens when you ask for the impossible

winlspci does not silently do less than you asked. A request that falls in the right-hand column above produces an explanation and a non-zero exit code:

```
PS> lspci -x
lspci: cannot dump configuration space (-x). Windows exposes no userland path
to PCI config space; that needs a signed kernel-mode driver. ...
```

The command exits with status **2**. A script that assumed `-x` would work therefore fails loudly at the point of the mistaken assumption, rather than quietly producing an inventory with a hole in it.

PRACTICAL READING

If your task is *presence, identity, binding, link state and topology*, winlspci covers it completely. If your task requires reading a register on the device right now — a capability walk, an ASPM state, an AER error log — no driverless tool can help, and you need a kernel-mode utility from your silicon vendor.

3 Requirements and installation

There is no installer, no service, and no driver. Clone the repository and run it.

3.1 Requirements

Operating system	Windows, with the standard PnP/PCI enumeration available (that is, a normally configured system)
Shell	Windows PowerShell 5.1 — the version that ships with Windows. No separate runtime to install.
Network	Not required. The PCI ID database is bundled.
Kernel driver	None. That is the point of the tool.

WHY 5.1 SPECIFICALLY

winlspci is written to Windows PowerShell 5.1 syntax on purpose — no ternary operator, no `??` null-coalescing, no `-AsHashtable`. The reasoning is simple: **a tool that tells you what is in the machine should not need an install before it runs**. Any Windows box you sit down at can run it as-is.

3.2 Clone and run

```
PS> git clone <this repo> C:\tools\winlspci
PS> $env:PATH += ';C:\tools\winlspci\bin'
PS> lspci -nn
```

That gives you the `lspci` command for the current session only. To make it permanent, append to the **User** PATH:

```
PS> $p = [Environment]::GetEnvironmentVariable('PATH','User')
PS> [Environment]::SetEnvironmentVariable('PATH', "$p;C:\tools\winlspci\bin", 'User')
```

DO NOT USE SETX

Use the .NET API shown above, **not** `setx`. The `setx` command silently truncates PATH at 1024 characters, and a developer machine is routinely well past that length. Truncating PATH destroys most of your environment with no warning and no error message.

3.3 Importing the module directly

If you would rather work with the cmdlets than the `lspci` command wrapper, import the module and call them:

```
PS> Import-Module C:\tools\winlspci\winlspci.psd1
PS> Get-PciDevice -Device '10de:' | Format-Lspci -Verbosity 2
```

The cmdlets return real PowerShell objects, so everything downstream of them — `Where-Object`, `Select-Object`, `Export-Csv`, comparison operators on numeric fields — behaves the way you expect. Chapter 7 builds on this.

3.4 Verifying the installation

```
PS> lspci -nn          # should print one line per device
PS> lspci -ListAttributes # should print the queryable attribute names
PS> (lspci).Count      # device count — a sanity check against Device Manager
```

4 Quick start

Five commands that cover most of what the tool is used for.

4.1 List everything, with names and IDs

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI Express Root Port #6
[8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold P31/BC711/PC711 NVMe SSD
[1c5c:174a] (rev 00)
```

4.2 Look closely at one vendor's devices

```
PS> lspci -d 1c5c: -vv
01:00.0 Non-Volatile memory controller: SK hynix Gold P31/BC711/PC711 NVMe Solid State Drive
(rev 00)
    LnkSta: 8GT/s x4 (max 8GT/s x4)
    Driver: stornvme (10.0.26100.8972)
    Subsystem: 174a1c5c
    DevCtl: MPS 256 bytes (max 512), MaxReadReq 512 bytes
    Capabilities: AER present
```

Note the **LnkSta** line: the negotiated speed and width, and in parentheses the maximum the link is capable of. Those two numbers agreeing is the answer to most link questions; them disagreeing is the beginning of an investigation.

4.3 See the tree

```
PS> lspci -t
-00:1d.0 Tiger Lake-LP PCI Express Root Port #9
 \-f3:00.0 GA107M [GeForce RTX 3050 Ti Mobile] (8GT/s x4)
```

4.4 Find fast links — without grep

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' # no grep needed
```

4.5 Find anything running below its maximum

```
PS> lspci -Downtrained # anything below its max
```

INTERPRETING -DOWNTRAINED

A downtrained link is a *finding*, not automatically a fault. On a laptop, a GPU reporting 8GT/s x4 against a maximum of 16GT/s x16 is almost certainly link power management at idle. winlspci tells you what the link is doing; deciding whether that is a problem is still yours.

5 Command reference — standard lspci flags

Measured against Linux `lspci`, not against an aspiration. Three lists follow: what is implemented, what is refused on principle, and what simply has not been built.

5.1 Implemented

LSPCI	WINLSPCI	NOTES
<code>-s <bdf></code>	<code>-s</code>	Prefix match. Accepts padded, unpadded and domain-qualified forms: <code>1:</code> , <code>01:</code> , <code>1:0.0</code> , <code>0000:01:00.0</code>
<code>-d <ven>:<dev>:<class></code>	<code>-d</code>	Any field may be left empty: <code>-d 11f8:</code> , <code>-d :174a</code> , <code>-d ::0108</code>
<code>-t</code>	<code>-t</code>	Real topology, built from <code>DEVPKEY_Device_Parent</code> . Shows link state inline
<code>-v</code>	<code>-v</code>	Link state (current and maximum), driver, device status
<code>-vv</code>	<code>-vv</code>	Adds MPS, MRRS, subsystem, NUMA node and AER presence
<code>-vvv</code>	<code>-vvv</code>	Every property Windows exposes, plus an explicit list of what is missing
<code>-n</code>	<code>-n</code>	Numeric IDs only
<code>-nn</code>	<code>-nn</code>	Names and IDs together
<code>-D</code>	<code>-Domain</code>	Not spelled <code>-D</code> : PowerShell matches parameter aliases case-insensitively , so <code>-D</code> would collide with <code>-d</code> . The value is always <code>0000</code> — Windows' PnP data carries no segment number
<code>-k</code>	shown by <code>-v</code>	Driver name and version are part of <code>-v</code> output

5.2 Deliberately refused

LSPCI	WHY
<code>-x</code> , <code>-xxx</code> , <code>-xxxx</code>	Configuration-space dumps. Impossible without a signed kernel-mode driver. <code>winlspci</code> exits 2 with an explanation rather than pretending otherwise.

5.3 Not implemented (possible, just not built)

These are absent by omission rather than by principle. Nothing about the driverless approach prevents them.

FLAG	PURPOSE / STATUS
-b	Bus-centric view
-m, -mm	Machine-readable output — superseded here by <code>-Json</code> , <code>-Csv</code> and <code>-Attribute</code>
-p	Custom ID file
-i	Custom <code>pci.ids</code> path
-q, -Q	Online ID lookup
-A, -H1, -H2, -F	Access-method selection — meaningless in a driverless implementation
-M	Bus mapping

6 Command reference — flags beyond lspci

These flags have no Linux counterpart. They exist because PowerShell's object pipeline makes possible what a text pipeline cannot do safely.

FLAG	WHY IT EXISTS
<code>-Attribute <names></code>	Query at the attribute level rather than the device level. Wildcards are accepted: <code>-Attribute Link*</code>
<code>-Match <regex></code>	Filter by value — the job you would otherwise have handed to <code>grep</code>
<code>-PresentOnly</code>	Drop attributes the device does not report, leaving only real data
<code>-ListAttributes</code>	Discover what is queryable on this build
<code>-Delimited</code>	<code> </code> -separated records, for <code>-split</code> / <code>cut</code> / <code>awk</code> habits
<code>-Delimiter</code>	Change the separator — for example a tab, giving TSV
<code>-Header</code>	Name the columns in delimited output
<code>-Json</code>	Structured JSON output
<code>-Csv</code>	Structured CSV output, correctly quoted
<code>-Downtrained</code>	Show only devices running below their maximum speed or width

CHOOSING BETWEEN THEM

Reach for `-Attribute` and `-Match` when you want a specific answer, `-Csv` or `-Json` when something else will consume the output, and `-Delimited` when you already know the one-liner you want to write.

7 Attribute-level queries

Windows has no `grep`, so `lspci -vv | grep LnkSta` has no equivalent. Rather than bolt a text search onto the tool, winlspci serialises the data to one record per attribute — which turns out to be strictly better than the thing it replaces.

7.1 One record per attribute

```
PS> lspci -Attribute Link* -s 01:00.0 -PresentOnly
```

Slot	VendorId	DeviceId	Attribute	Value	Present
01:00.0	1c5c	174a	LinkStateReported	True	True
01:00.0	1c5c	174a	LinkSpeed	8GT/s	True
01:00.0	1c5c	174a	LinkWidth	4	True

7.2 Filtering by value

This is the part you would have piped to `grep`:

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' # every Gen4+ link
PS> lspci -Attribute Driver -Match 'stornvme'           # everything on one driver
PS> lspci -Attribute '*' -Match '11f8' -Csv            # every field mentioning the vendor
PS> lspci -ListAttributes                             # what can I ask for?
```

7.3 Three properties that beat text search

Present is a field

Absent and zero never collapse into each other. A root port that reports no link width is not a device running at x0 — and a text pipeline cannot tell those apart.

Values keep their types

`LinkWidth` is an integer, so `-lt 4` is a numeric comparison. `grep` would have you comparing strings.

No match exits 1

An attribute query that matches nothing returns a non-zero exit code, so a script can tell “no match” from “it worked”.

7.4 Composing with the pipeline

Because these are real objects, ordinary PowerShell applies:

```
# every device not running at full width, as a table
Get-PciDevice | Where-Object { $_.LinkStateReported -and $_.LinkWidth -lt $_.MaxLinkWidth } |
    Select-Object Slot, DeviceName, LinkWidth, MaxLinkWidth

# diff two machines
Get-PciDevice | ConvertTo-PciAttributeRecord | Export-Csv machine-a.csv
```

Exporting the attribute records from two systems and diffing the two CSV files is the fastest way to answer “what is different about this machine?” — a question that is otherwise a long afternoon.

8 Delimited output and coming from Linux

You cannot have `grep`, `awk` and `cut` here, but you can have the shape of them.

8.1 Delimited records

`-Delimited` emits one record per line with `|` between fields:

```
PS> lspci -Delimited -s 01:
01:00.0|1c5c|174a|0108|Non-Volatile memory controller|SK hynix|Gold P31 NVMe
SSD|00|86T/s|4|86T/s|4|stornvme|OK
```

`-Header` names the columns; `-Delimiter "`t"` gives you TSV.

8.2 Translation table

LINUX	WINLSPCI
<code>lspci grep NVMe</code>	<code>lspci Select-String NVMe</code>
<code>lspci -vv grep LnkSta</code>	<code>lspci -Attribute LinkSpeed,LinkWidth</code>
<code>lspci cut -d' ' -f1</code>	<code>lspci -Delimited %{ (\$_ -split '\ ')[0] }</code>
<code>lspci awk -F' ' '{print \$1, \$9}'</code>	<code>lspci -Delimited %{ \$f=\$_ -split '\ '; "\$(\$f[0]) \$(\$f[8])" }</code>
<code>lspci wc -l</code>	<code>(lspci).Count</code>
<code>lspci -d 11f8: echo "absent"</code>	Same — a filter matching nothing exits 1

8.3 Two properties worth relying on

An empty field means “not reported”; a literal `0` means zero

The distinction the object model protects survives into the text form for free, because an unset value serialises to nothing at all. This is real output from the machine `winlspci` was written on:

```
00:1e.0|8086|a0a8|0780|Communication controller|Intel Corporation|Tiger Lake-LP Serial IO UART
Controller #0|20|||||iaLPSS2_UART2_TGL|OK
    ^ the four consecutive empty fields above are the link-state columns: not reported
f3:00.0|10de|25a0|0302|3D controller|NVIDIA Corporation|GA107M [GeForce RTX 3050 Ti
Mobile]|a1|86T/s|4|166T/s|16|nvlddmkm|OK
```

A device with no link state and a device genuinely training at x0 are different findings, and `grep`-shaped output normally destroys that difference. Here it survives a `-split`.

THE SECOND LINE, READ CLOSELY

That GPU reports **8GT/s x4 against a maximum of 16GT/s x16** — which is exactly why `-Downtrained` exists. On a laptop it is almost certainly link power management at idle rather than a fault, and that is the point: the tool tells you what the link is doing, and the judgement stays with you.

The delimiter is stripped from values, not escaped

Quoting rules turn a one-liner into a parser, which defeats the purpose — so a `|` appearing inside a value becomes `/` and the column count stays fixed. No `|` occurs in live PCI data or in `pci.ids` today, but `FriendlyName` comes from driver INF files and is not under the tool's control.

IF YOU NEED REAL QUOTING

Use `-Csv`. It quotes properly. `-Delimited` is a convenience for known-shaped pipelines, not a general-purpose serialisation format.

STRUCTURED BEATS TEXTUAL WHERE IT CAN

`lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' keepstypes`, so `LinkWidth -lt 4` is a numeric comparison rather than a string one. Reach for `-Delimited` when you already know the pipeline you want to write.

9 Topology

`-t` builds a real tree from `DEVPKEY_Device_Parent`, so an endpoint appears under the root port that actually carries it — with its link state inline, which is where a downtrained device becomes obvious at a glance.

```
-00:06.0 11th Gen Core Processor PCIe Controller
 \-01:00.0 Gold P31/BC711/PC711 NVMe Solid State Drive (8GT/s x4)
-00:1d.0 Tiger Lake-LP PCI Express Root Port #9
 \-f3:00.0 GA107M [GeForce RTX 3050 Ti Mobile] (8GT/s x4)
```

9.1 Reading the tree

- A line beginning `-` at the left margin is a root of the tree — typically a root port or host bridge.
- `\-` marks a child of the entry above it.
- The value in parentheses is the endpoint's current link state, in *speed × width* form.

9.2 Devices with a non-PCI parent

A device whose parent is not itself a PCI device becomes a root of the tree rather than being dropped from it.

WHY THAT CHOICE

An incomplete tree that *looks* complete is worse than an obviously ragged one. A test asserts that every enumerated device appears in the tree exactly once, so what you see is the whole inventory even when the hierarchy is untidy.

10 The PCI ID database

Vendor and device names come from `pci.ids`, maintained by the PCI ID Project. `winspci` bundles a copy so that it works with no network — which, on a lab bench, is usually the situation.

Bundled location	<code>data/pci.ids</code>
Approximate size	~1.4 MB
Source	The PCI ID Project — https://pci-ids.ucw.cz
Licence	BSD / GPL dual-licensed
Network needed to run	No

10.1 Refreshing the database

Updates are explicit — the tool never fetches in the background:

```
PS> Update-PciIds           # from https://pci-ids.ucw.cz
```

The update refuses to replace a working database with a suspiciously short download. A truncated fetch over a captive-portal network would otherwise leave you with a tool that reports numeric IDs and no names, at the exact moment you need names.

10.2 When a name looks wrong

VENDOR ID 11F8 READS “PMC-SIERRA INC.”

Microsemi acquired PMC-Sierra; Microchip acquired Microsemi; the vendor ID never changed. A Microchip Switchtec switch therefore shows as **PMC-Sierra**. There is a test pinning this behaviour, so that nobody at a bench at 11pm concludes they have the wrong part.

The general lesson holds beyond this one case: a vendor name in `pci.ids` records who registered the ID, not who sells the silicon today. When a name surprises you, check the acquisition history before you check the hardware.

11 Design principles

Three decisions shape how winlspci behaves in the cases that matter. All three come from the same instinct: **a wrong answer that looks right is worse than an obvious failure.**

11.1 “Not reported” is never rendered as zero

Root ports and chipset devices legitimately expose no link state. Rendering that as `x0` would look like a dead link — a false alarm that costs somebody an hour. So an absent value stays `$null` internally and prints as “*not reported by this device*”. `Present` is a real field in attribute output for the same reason. There is a test for it.

11.2 A filter that matches nothing exits non-zero

`lspci -d 11f8:` finding no card must not look like success. That is the difference between “*the card is not there*” and “*the command worked*”, and a build script needs to be able to tell them apart. The same rule applies to an attribute query that returns no rows.

11.3 Slot filters normalise both sides

`-s 1:` once returned *nothing* while `-s 01:` matched the same device, because the comparison ran against the raw string. A filter that silently finds nothing when the device is right in front of you is the worst failure this tool could have.

Padded, unpadded and domain-qualified forms are therefore all normalised before comparison, and tests assert that `1:` and `01:` agree.

Absent ≠ zero

Missing data prints as missing, never as a value that looks like a fault.

Empty ≠ success

No match is a non-zero exit, so automation can branch on it correctly.

Input is normalised

Every reasonable spelling of a slot address matches the same device.

12 Performance

Full enumeration of a laptop (24 devices) takes roughly two seconds; a filtered query around one second.

~2_s

Full enumeration, 24 devices

~1_s

Filtered query

~39_{ms}

Per device, winning method

~1230_{ms}

Per device, cmdlet path

12.1 How it got there

Reaching those numbers took three attempts, and the middle one is worth recording.

APPROACH	TIME
<code>Get-PnpDevice</code> + per-property <code>Get-PnpDeviceProperty</code>	> 5 min
Batched <code>Get-PnpDeviceProperty -KeyName <16 keys></code>	~36 s
<code>Get-PnpDeviceProperty -KeyName '*'</code>	<i>appeared 2× faster – returned zero properties</i>
<code>Win32_PnPEntity</code> + <code>Invoke-CimMethod</code> <code>GetDeviceProperties</code>	~2 s

THE INTERESTING FAILURE

The wildcard looked like the fastest option and was in fact doing nothing, because `-KeyName` does not support wildcards. The benchmark was measuring a no-op, and the output still looked plausible. Only checking the *values* caught it — a reminder that a performance test which does not assert on results is measuring the wrong thing.

12.2 Why the winning approach wins

The final implementation calls the CIM method that the cmdlet wraps: roughly **39 ms per device versus ~1230 ms**. The cmdlet appears to re-resolve the device on every call, while the CIM method takes an already-resolved instance and works from it.

13 Exit codes and scripting

`winlspci` is built to be used from scripts as well as by hand, which means the exit code carries meaning. This chapter summarises the behaviour described throughout the manual.

CODE	MEANING	WHEN YOU SEE IT
0	Success	The command ran and produced at least one matching result
1	No match	A device filter (<code>-s</code> , <code>-d</code>) or an attribute query matched nothing. The command itself worked — the hardware is simply not there
2	Refused / not possible	You asked for something a driverless tool cannot do, such as <code>-x</code> . An explanation is printed

13.1 Using it in a script

The distinction between **1** and **2** is the useful one: exit 1 is a statement about the machine, exit 2 is a statement about your command.

```
lspci -d 11f8: | Out-Null
if ($LASTEXITCODE -eq 0) { 'switch present' }
elseif ($LASTEXITCODE -eq 1) { 'switch absent' }
else { 'query could not be answered' }
```

The *semantics* are the same as on Linux, so a colleague's `lspci -d 11f8: || echo "absent"` translates directly — but write the test with `$LASTEXITCODE` as above rather than with `||`, which Windows PowerShell 5.1 does not support.

STRUCTURED OUTPUT IN AUTOMATION

For anything that another program will parse, prefer `-Json` or `-Csv` over `-Delimited`: both quote correctly and neither depends on a fixed column order staying fixed.

14 Troubleshooting

The situations below follow directly from how the tool works. Most of them are the tool being correct rather than the tool being broken.

SYMPTOM	CAUSE AND RESOLUTION
<code>-s 1:</code> and <code>-s 01:</code> seem to disagree	They should not — both forms are normalised before comparison, and a test pins that behaviour. If you observe a difference, it is a defect worth reporting.
A device shows no link speed or width	Not a fault. Root ports and chipset devices legitimately expose no link state; the field is <i>not reported</i> , which is deliberately distinct from a value of zero. See §11.1.
A device reports less than its maximum speed or width	Real, but not automatically a problem — link power management routinely lowers both at idle. Use <code>-Downtrained</code> to list every such device, then judge in context.
A vendor name looks wrong (e.g. a Microchip part reads PMC-Sierra)	Correct behaviour. <code>pci.ids</code> records who registered the vendor ID, not who owns the product line today. See §10.2.
<code>lspci -x</code> refuses to run	By design. Configuration-space dumps need a signed kernel-mode driver. Exit code 2. See chapter 2.
A filter returns nothing and the script treats it as failure	Intended: no match is exit 1, not exit 0. Branch on <code>\$LASTEXITCODE</code> rather than on empty output. See chapter 13.
<code>lspci</code> is not recognised as a command	The <code>bin</code> directory is not on PATH for this session. Re-run the PATH line from §3.2, or import the module directly per §3.3.
PATH additions vanished, or the environment broke after setting PATH	Almost certainly <code>setx</code> , which truncates PATH at 1024 characters without warning. Use the .NET API shown in §3.2.
Devices show numeric IDs but no names	The <code>pci.ids</code> database is missing or damaged. Run <code>Update-PciIds</code> ; the update refuses a suspiciously short download rather than overwriting a good file.
A parsed one-liner produces the wrong column	The delimiter is stripped from values, not escaped, so column count is fixed — but an <i>empty</i> field is legitimate and means “not reported”. If your parser collapses empty fields, use <code>-Csv</code> instead. See §8.3.

15 Tests

The test suite runs from a stock Windows install with nothing added.

```
PS> powershell -ExecutionPolicy Bypass -File tests\Invoke-Tests.ps1
```

15.1 Why not Pester

No Pester, deliberately. Stock Windows ships Pester 3.4.0, whose `Should Be` syntax is incompatible with Pester 5's `Should -Be`. A Pester suite would therefore pass on the author's machine and fail on a colleague's — precisely the class of misleading result the rest of the tool is built to avoid.

15.2 Hardware-dependent tests

The PCI inventory differs on every machine, so hardware-dependent tests assert on *shape* rather than on values. Where a test genuinely needs a specific device present, it skips with a stated reason rather than failing or, worse, passing vacuously.

WHAT THE SUITE PINS

Among others: that absent values never render as zero (§11.1); that `1:` and `01:` resolve identically (§11.3); that every enumerated device appears in the topology tree exactly once (§9.2); and that vendor `11f8` resolves to PMC-Sierra (§10.2).

A Appendix A — Flag quick reference

One page to keep next to the bench.

Selection

<code>-s <bdf></code>	Select by slot address; prefix match; padded, unpadded and domain-qualified all accepted
<code>-d <ven>:<dev>:<class></code>	Select by ID; any field may be empty
<code>-Downtrained</code>	Only devices below their maximum speed or width

Verbosity and format

<code>-n / -nn</code>	IDs only / names and IDs
<code>-v / -vv / -vvv</code>	Link state, driver, status / adds MPS, MRRS, subsystem, NUMA, AER / everything, plus what is missing
<code>-t</code>	Topology tree with inline link state
<code>-Domain</code>	Show the domain field (always <code>0000</code>)
<code>-Json / -Csv</code>	Structured output
<code>-Delimited</code>	<code> </code> -separated records
<code>-Delimiter / -Header</code>	Change the separator / name the columns

Attribute queries

<code>-Attribute <names></code>	One record per attribute; wildcards accepted
<code>-Match <regex></code>	Filter by value
<code>-PresentOnly</code>	Drop attributes the device does not report
<code>-ListAttributes</code>	List what is queryable

Cmdlets

<code>Get-PciDevice</code>	Enumerate devices as objects
<code>Format-Lspci</code>	Render device objects in <code>lspci</code> style; takes <code>-Verbosity</code>
<code>ConvertTo-PciAttributeRecord</code>	Expand device objects to one record per attribute
<code>Update-PciIds</code>	Refresh the bundled <code>pci.ids</code> database

Refused / absent

<code>-x, -xxx, -xxxx</code>	Refused — needs a signed kernel-mode driver; exits 2
<code>-b, -m, -mm, -p, -i, -q, -Q, -A, -H1, -H2, -F, -M</code>	Not implemented

B Appendix B — Glossary

AER	Advanced Error Reporting. A PCI Express capability for logging and reporting link and transaction errors. winlspci reports whether the capability is <i>present</i> ; the register contents require a live read and are out of reach.
------------	---

BDF	<code>bus:device.function</code> — the address that identifies a PCI function. Written <code>01:00.0</code> , optionally prefixed with a domain: <code>0000:01:00.0</code> .
Configuration space	The per-function register space that holds PCI identity and capability structures. Reading it directly from userland is not possible on Windows — the reason winlspci works the way it does.
Domain / segment	The highest-order part of a PCI address. Windows' PnP data carries no segment number, so winlspci always reports <code>0000</code> .
Downtrained	A link operating below its maximum speed, width, or both. Often power management rather than a fault.
GT/s	Gigatransfers per second — the PCIe signalling rate. 8GT/s is Gen3, 16GT/s Gen4, 32GT/s Gen5, 64GT/s Gen6.
Link width	The number of lanes in use, written x1, x4, x8, x16. Reported both as negotiated and as maximum.
MPS	Max Payload Size — the largest data payload the device is configured to move in one transaction.
MRRS	Max Read Request size — the largest read the device will issue in a single request.
NUMA node	The memory/CPU locality domain a device is attached to. Relevant to bandwidth and latency on multi-socket systems.
pci.ids	The community-maintained database mapping numeric PCI IDs to vendor and device names. Bundled with winlspci; refreshed with <code>Update-PciIds</code> .
PnP	Plug and Play — the Windows subsystem that enumerates devices. winlspci reads its data, populated by the PCI bus driver from configuration space at enumeration time.
Prog-if	Programming interface — the third component of the PCI class code, after class and subclass.
Root port	The PCIe port on the host side of a link. Frequently reports no link state of its own, which winlspci renders as <i>not reported</i> rather than zero.
Subsystem ID	Vendor and device IDs assigned by the board integrator, distinguishing one implementation of a chip from another.

C Appendix C — Licence and credits

Licence

winlspci is released under the **MIT Licence**.

The bundled PCI ID database, `data/pci.ids`, is **BSD / GPL dual-licensed** and comes from the PCI ID Project. It is redistributed under those terms and is not covered by the MIT licence above.

Credits

pci.ids

Maintained by the PCI ID Project — <https://pci-ids.ucw.cz>

lspci

The original Linux utility, part of the pciutils package, whose interface winlspci deliberately follows

Scope of this document

ABOUT THIS MANUAL

This manual documents the behaviour of winlspci as released. Chapters 13, 14 and Appendix B organise and expand on behaviour described elsewhere in the documentation set; they introduce no flags, cmdlets or output formats beyond those listed in chapters 5 and 6. Where the tool's capabilities are bounded by Windows itself, chapter 2 is the authoritative statement of that boundary.



WINLSPCI — POWERSHELL PCI TOOL (NO DRIVER)

Document revision 1.0 · August 2026 · Specifications subject to change